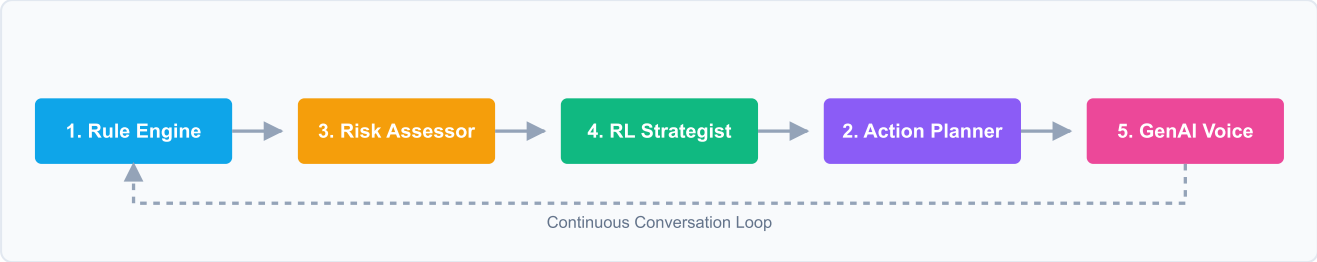
Implementation Blueprint

Policy-Adherent Agent Data Flow & Module Integration

This document defines the exact execution pipeline for building the Policy-Adherent Agent. It breaks down the system into a sequential flowchart, detailing the specific responsibilities of each module, what code you need to write, and how data must transition between components.



Module Dependencies Map

Module 1: Search & Constraints

The logical brain. Maps requests to policies (CSP) and calculates tactical defense dialogue (Minimax).

Module 3: Uncertainty

The risk evaluator. Uses Bayes theorem to calculate the probability of fraud and evidence reliability.

Module 4: Reinforcement Learning

The grand strategist. Selects the ultimate goal (approve/reject/investigate) based on M1 and M3 states.

Module 2: Planning

The safety sequencer. Ensures irreversible actions (like refunds) only occur after verification steps.

Module 5: Generative AI

The voice. Translates the strict tactical logic of M1 into natural, empathetic human conversation.

Phase 1: Ingestion & Rule Evaluation

Step 1: Ingestion & Extraction

Module 4 (NLU)

What you must code: A script (likely utilizing an LLM prompt) that takes the raw customer chat string and extracts structured JSON. It must isolate claims, dates, provided evidence, and categorize the emotional tone or manipulative strategies being used.

Input (from User)	Output (to Module 1)
"My $500 TV is broken, refund me now or I'll sue!"	JSON: `{"intent": "refund", "amount": 500, "threat_level": "high"}`



Step 2: Constraint Satisfaction Problem (CSP)

Module 1

What you must code: A Python script using a library like `python-constraint` or a custom graph evaluator. It maps the JSON from Step 1 against your loaded policy graph. For every rule required for a refund, it checks if the data exists. It must explicitly tag constraints that are missing or suspicious.

Input (from Step 1)	Output (to Module 3 & 4)
Structured claim JSON + Policy Graph definitions.	Constraint variables tagged as: `Satisfied`, `Violated`, or **`Uncertain`**.

Developer Checkpoint: If all constraints are `Satisfied`, the system can jump directly to execution. If any constraint is `Uncertain` (e.g., missing a photo of the TV), the flow MUST proceed to Step 3.

Phase 2: Risk Assessment & Strategy

Step 3: Reasoning Under Uncertainty

Module 3

What you must code: A probabilistic model (e.g., a Bayesian Network using `pgmpy`). It consumes the `Uncertain` flags from Module 1. It calculates a mathematical probability of whether the customer is lying, based on the weakness of the evidence and the cost of the item.

Input (from Step 2)	Output (to Module 4)
`Uncertain` constraints + Evidence metadata.	Numerical scores: e.g., `fraud_prob: 0.78`, `evidence: weak`.



Step 4: Reinforcement Learning Decision

Module 4

What you must code: An RL environment (using Q-learning or Contextual Bandits). The "State" is a combination of the CSP flags (Module 1) and the Fraud Probability (Module 3). You must define a Reward Function that penalizes financial loss and policy breaches. The agent selects the highest-reward action.

Input (State)	Output (Action Directive)
State vector: `[missing_photo, fraud=0.78, VIP=no]`	Action integer mapping to: `ASK_FOR_EVIDENCE`.

Developer Checkpoint: Module 4 is the commander. It does not write the chat message. It simply decides *what the system's goal is* for the current turn. This goal is now passed down to the tactical modules to execute safely.

Phase 3: Execution & Generation

Step 5: Sequential Planning

Module 2

What you must code: A simple planning script. Before executing Module 4's action, this checks safety preconditions. If Module 4 decided to "Approve Refund", Module 2 verifies that the "Item Returned" step has actually occurred in the sequence.

Step 6 & 7: Minimax Defense and GenAI

Module 1 & 5

What you must code: A Minimax search tree with Alpha-Beta pruning (Module 1). The Agent tries to maximize compliance; the simulated user tries to minimize it. The algorithm evaluates dialogue options to achieve Module 4's goal without triggering the user's threats. Module 5 (GenAI) then takes the winning path and generates natural text.

Input (from Step 4)	Output (to Customer)
Goal: `ASK_FOR_EVIDENCE` + Threat: Sue	"I understand your frustration. To process this, policy requires a photo. Can you provide one?"

